Recommendation Systems and Bandits

Memory, prediction, and the difference between them

Carlos Cotrini

ETH Zürich | AI in Industry

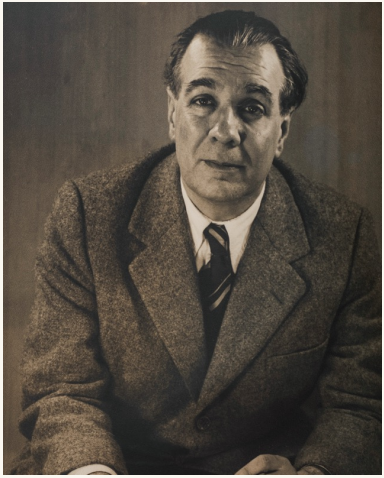
Week 2, Spring 2026

ETH ZÜRICH

Companion website

The ten interactive demos used in the lecture live here, numbered in the order in which we will visit them.

eth-ainit-fs26.github.io/recsys_bandits_lecture



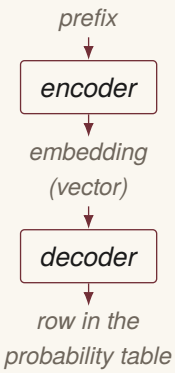
"To think is to forget differences, generalize, abstract."

Jorge Luis Borges, *Funes the Memorious*, 1942

Last week, in language

An LLM ≈ encoder (builds meaning) + decoder (continues the sentence).

The *encoder* reads a prefix and produces an *embedding*: a vector summarising what the prefix says. The *decoder* takes that vector and produces from it the corresponding row in the probability table.



Two halves of one machine

The encoder and decoder have older names.

- Encoder ≈ *compression*. A long sequence becomes a short vector.
- Decoder ≈ *prediction*. The vector becomes a bet about what comes next.

This framework is the core of many modern machine learning technologies, and originates from a 1948 paper by Shannon.

The promise

Machine learning is the engineering of two old theorems. *Compression*, the shortest faithful description of the past, and *prediction*, the best bet about the future, are two views of the same subject, governed by Shannon's 1948 theorems. We will build them in two halves and collapse them in the third.

I

Compression

"... a Map of the Empire whose size was that of the Empire, and which coincided point for point with it."

Jorge Luis Borges, *On Exactitude in Science*, 1946

The data: 21 readers, 16 books

A small public library has kept track of 21 readers and 16 books. Each cell records whether the reader liked the book, disliked it, or has not yet read it.

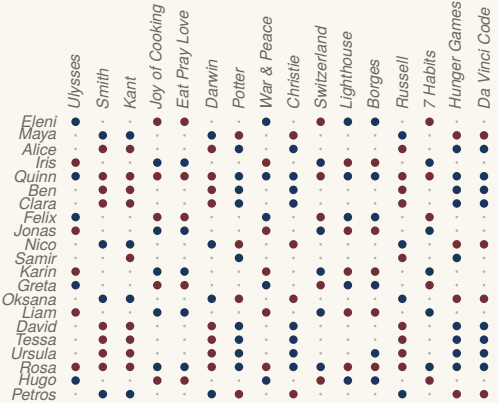
We want to extract insights from this matrix.

Sixteen books

Each book is referred to by its *short label* (right column).

The Wealth of Nations	Adam Smith, 1776	*Smith*
Critique of Pure Reason	Immanuel Kant, 1781	*Kant*
On the Origin of Species	Charles Darwin, 1859	*Darwin*
War and Peace	Leo Tolstoy, 1869	*War & Peace*
Ulysses	James Joyce, 1922	*Ulysses*
To the Lighthouse	Virginia Woolf, 1927	*Lighthouse*
The Joy of Cooking	Irma Rombauer, 1931	*Joy of Cooking*
Murder on the Orient Express	Agatha Christie, 1934	*Christie*
Ficciones	Jorge Luis Borges, 1944	*Borges*
A History of Western Philosophy	Bertrand Russell, 1945	*Russell*
The 7 Habits of Highly Effective People	Stephen Covey, 1989	*7 Habits*
Harry Potter and the Philosopher's Stone	J. K. Rowling, 1997	*Potter*
The Da Vinci Code	Dan Brown, 2003	*Da Vinci Code*
Eat, Pray, Love	Elizabeth Gilbert, 2006	*Eat Pray Love*
The Hunger Games	Suzanne Collins, 2008	*Hunger Games*
Lonely Planet: Switzerland	travel guide, 2024	*Switzerland*

Raw data



Three hundred and thirty-six cells. Can your brain remember them?

Now switching to:

Demo 1 / 10

Block-diagonal puzzle

open card Demo 1 on the companion site

Same trick, real data

Our 21 × 16 toy was *rank-two by construction.* Real ratings are noisy. Does the same sort still work?

Forty users and twenty-eight movies from the public *MovieLens 100K* dataset (1998). Four genre clusters (Family, Action, Drama, Comedy). Each cell is a real review, mean-centred per user.

Scrambled, the matrix looks like noise. Sorted by the same SVD-then-cluster trick, the blocks come back.

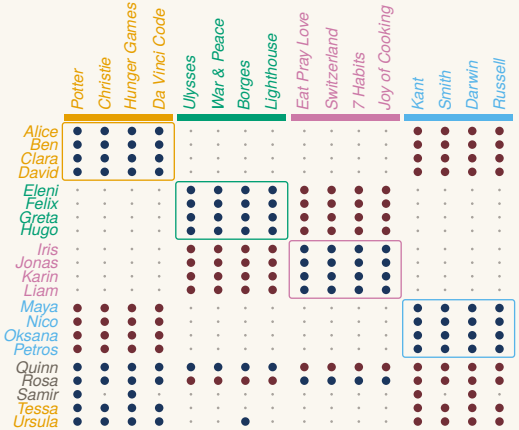
Now switching to:

Demo 2 / 10

MovieLens block-diagonal

open card Demo 2 on the companion site

Sort the readers, sort the books



Sort the rows, sort the columns, and four piles of books fall out.

Four piles of books. What is the pattern?

Potter	*Ulysses*	*Eat Pray Love*	*Kant*
Christie	*War & Peace*	*Switzerland*	*Smith*
Hunger Games	*Borges*	*7 Habits*	*Darwin*
Da Vinci Code	*Lighthouse*	*Joy of Cooking*	*Russell*

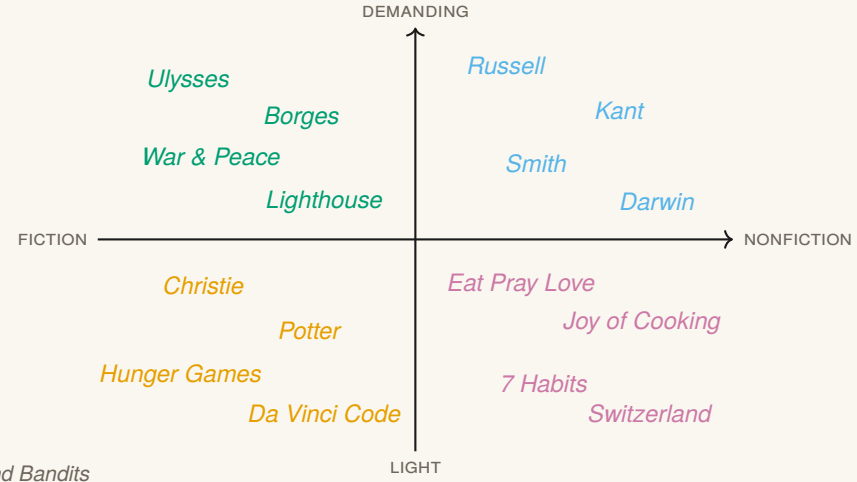
What two things does each pile share?

Sixteen books on one line



Sixteen books collapse to *two stacks*. Yet each stack carries *two colours*, so one axis is not enough.

Add a second axis. Each book finds its corner.

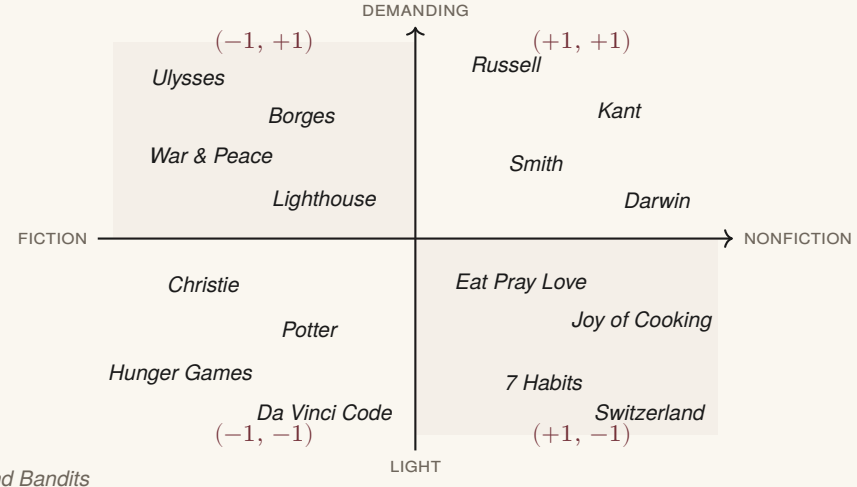


The geometry was always there

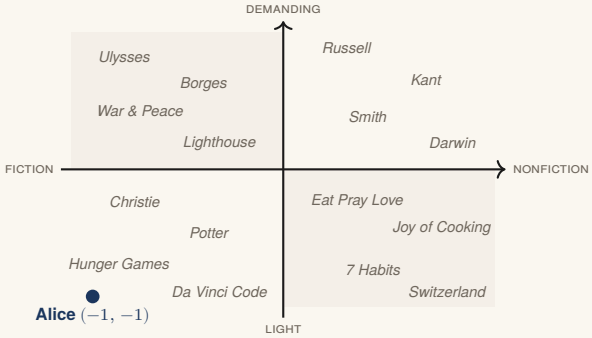
Two binary distinctions, four corners.

We did not impose these axes. We inferred them from a matrix of likes and dislikes alone.

Each book gets two numbers



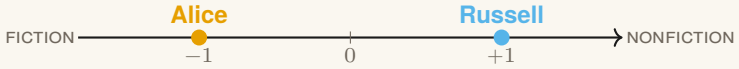
And readers, too



Readers live in the same plane: Alice, a light-fiction reader, sits among the light-fiction books.

One axis at a time

Forget light vs demanding for a moment. Just fiction vs nonfiction.

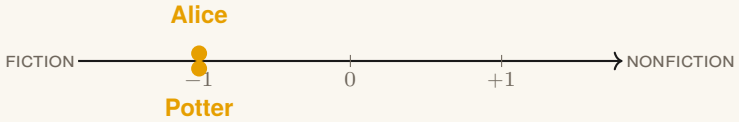


$$(-1) \times (+1) = -1$$

Different signs → -1 (a mismatch). Multiplication is a one-axis match score.

Same trick: Alice vs Potter

Same axis, different book.

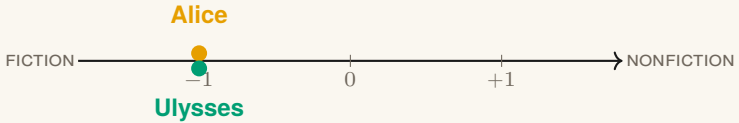


$$(-1) \times (-1) = +1$$

Same signs → $+1$ (a match). Both fiction; the axis agrees.

Same trick: Alice vs Ulysses

And again with Ulysses, a very different book.

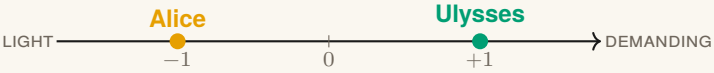


$$(-1) \times (-1) = +1$$

Both fiction, so another match on this axis, even though Ulysses and Potter are nothing alike.

Now the light axis: Alice vs Ulysses

Same multiplication, second axis. Alice prefers *light* reads (-1); Ulysses is *demanding* ($+1$).



$$(-1) \times (+1) = -1$$

On this axis they disagree. Alice and Ulysses agree on fiction but not on demand.

Two axes, two votes: Alice vs Ulysses

Two votes per pair. *Add* them.

	Alice	**Ulysses**
	light fiction	demanding fiction
	$(-1,\ -1)$	$(-1,\ +1)$
FICTION-AXIS VOTE:	$(-1) \times (-1) = +1$	
LIGHT-AXIS VOTE:	$(-1) \times (+1) = -1$	
MATCH SCORE:		0

? *A tie. Fiction matches; demand differs.*

Two axes, two votes: Alice vs Russell

Same recipe. Both votes go negative.

	Alice	Russell
	light fiction	demanding nonfiction
	$(-1,\ -1)$	$(+1,\ +1)$
FICTION-AXIS VOTE:	$(-1) \times (+1) = -1$	
LIGHT-AXIS VOTE:	$(-1) \times (+1) = -1$	
MATCH SCORE:		-2

× *Alice will not finish Russell.*

Two axes, two votes: Alice vs Potter

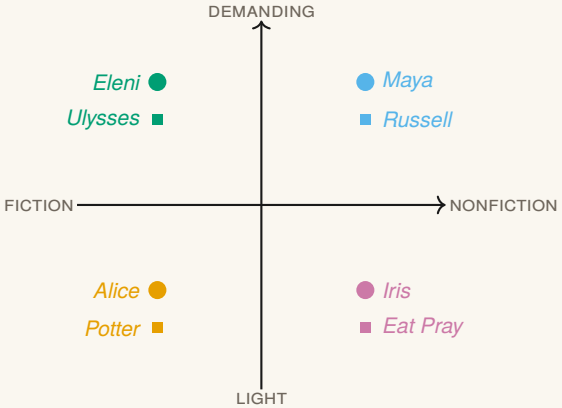
Same recipe. Both votes go positive.

	Alice	**Potter**
	light fiction	light fiction
	$(-1,\ -1)$	$(-1,\ -1)$
FICTION-AXIS VOTE:	$(-1) \times (-1) = +1$	
LIGHT-AXIS VOTE:	$(-1) \times (-1) = +1$	
MATCH SCORE:	$+2$	

✓ *Alice will love Potter.*

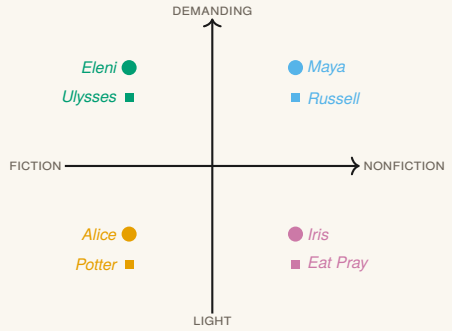
The matrix arose from the embeddings

Each cell of the matrix is the inner product of two points in the plane: a *reader* times a *book*.



The matrix arose from the embeddings

Each cell of the matrix is the inner product of two points in the plane: a *reader* times a *book*.



	Potter	Ulysses	Eat Pray	Russell
Alice	+2	0	0	−2
Eleni	0	+2	−2	0
Iris	0	−2	+2	0
Maya	−2	0	0	+2

Each matrix cell is one inner product of two points.

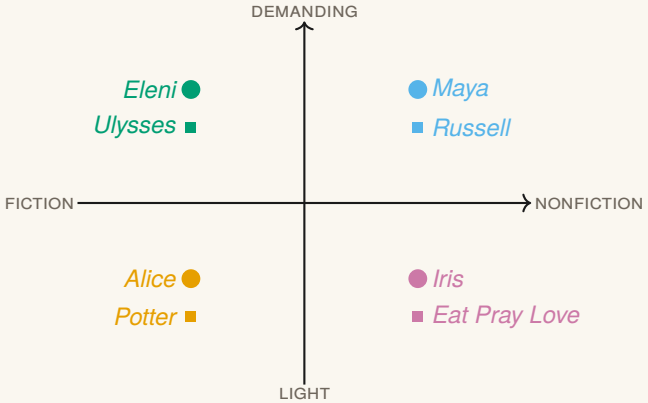
Same corner, opposite corner, sideways

READER \ BOOK	Potter	Ulysses	Eat Pray Love	Russell
Alice	$+2$	0	0	-2
Eleni	0	$+2$	-2	0
Iris	0	-2	$+2$	0
Maya	-2	0	0	$+2$

Same corner $\to +2$ (e.g. Alice $\times$ Potter). Opposite corner $\to -2$ (Alice $\times$ Russell). Side-by-side $\to 0$ (Alice $\times$ Ulysses).

Readers and books, same plane

Each reader *and* each book has two coordinates, and the same Cartesian plane carries them both.



Preference is a dot product

The *match score* we have been computing has a name: the *dot product*.

$$r = u \cdot v$$

where u is the embedding of a user (a reader) and v is the embedding of an item (a book).

Multiply on each axis. Add the products.

For Alice meeting Russell: $u_{\text{Alice}} \cdot v_{\text{Russell}} = (-1)(+1) + (-1)(+1) = -2.$

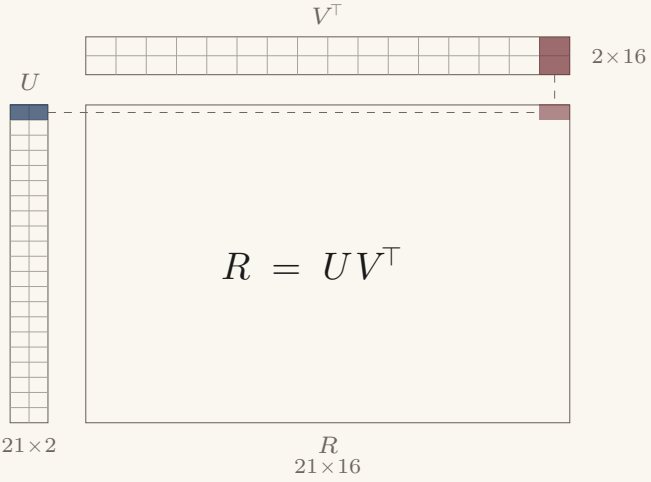
Two axes here, thousands in the wild

This lecture lives in 2 axes. Real recommenders live in tens to thousands. The formula does not change.

2 axes (today)	$u \cdot v = u_1 v_1 + u_2 v_2$
50 axes (a movie recommender)	$u \cdot v = u_1 v_1 + u_2 v_2 + \cdots + u_{50}\, v_{50}$
768 axes (word embeddings)	$u \cdot v = u_1 v_1 + \cdots + u_{768}\, v_{768}$
k axes (general)	$u \cdot v = u_1 v_1 + u_2 v_2 + \cdots + u_k\, v_k$

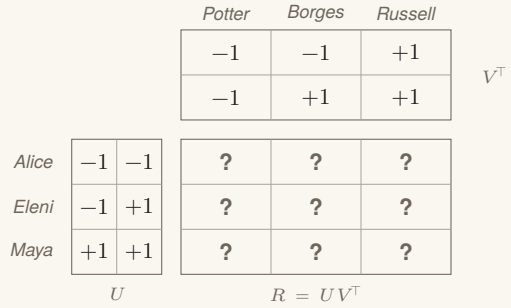
More axes, more terms. Same recipe: multiply on each axis, add.

Stack and multiply



The matrix product, in numbers

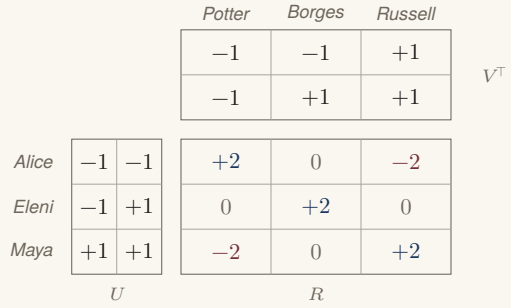
We have U (readers) and $V^\top$ (books). What is R?



For each cell of R: take the row of U, the column of $V^\top$, multiply on each axis, and add.

The matrix product, in numbers

Compute it cell by cell:



$+2$ on the diagonal (matched corners), -2 on the anti-diagonal (opposite corners), 0 where the axes disagree.

But we only had R

We never observed U or V directly. The library's ledger has only R, the rating matrix, three hundred and thirty-six cells of symbols.

Yet the matrix is the product of two skinny ones. So the question is:

Given R alone, can we recover U and V?

This is the *matrix factorization* problem. The next demo gives one way to solve it.

Now switching to:

Demo 3 / 10

Spectral microscope

open card Demo 3 on the companion site

Eckart & Young (1936)[1]

> *Among all rank-k approximations of any matrix, the one given by the top-k singular vectors is optimal in squared error.*

No algorithm can beat that. No future machine learning technology can beat it.

Now switching to:

Demo 4 / 10

Embeddings that learn

open card Demo 4 on the companion site

Further applications of matrix factorisation

The same trick recurs whenever you have a matrix. Two examples from ETH's *Statistical Machine Learning* course (FS26):

- *Eigenfaces.* Each row is a face, each column a pixel. The first few factors are the dominant axes of facial variation: lighting, pose, identity.

  sml-fs26.github.io/pca-kmeans-rf/eigenfaces-pca

- *The Congress story.* Each row is a legislator, each column a roll-call vote. The first two factors recover the well-known left/right and dovish/hawkish axes of U.S. politics, without anyone labelling the data.

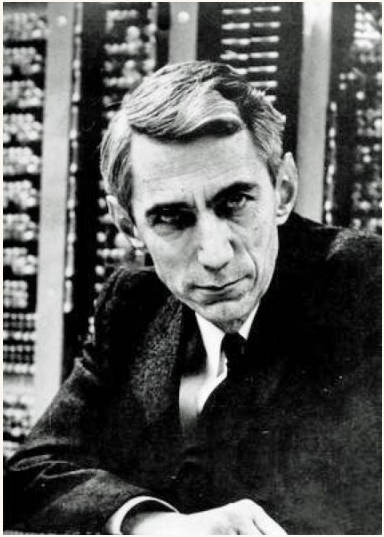
  sml-fs26.github.io/pca-kmeans-rf/congress-story

Faces, votes, ratings: different matrices, same factorisation.

End of Act I

We compressed a matrix. We named the act.

The names *entropy*, *rank*, and *factorization* were old words from 1936 and 1948.



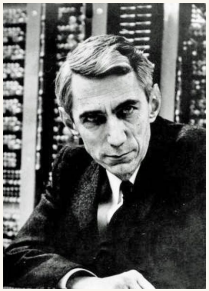
"The fundamental problem of communication is that of reproducing at one point either exactly or approximately a message selected at another point."

Claude E. Shannon, 1948

Interlude

Shannon, 1948

In 1948, Claude Shannon published *A Mathematical Theory of Communication*, a paper with two theorems at its core. The *source coding theorem* fixes a floor on how compactly any sequence of symbols can be described; the *channel coding theorem* fixes a ceiling on how reliably they can be transmitted through noise. For the next seven minutes we restate the first of these in the only vocabulary this lecture admits: urns.

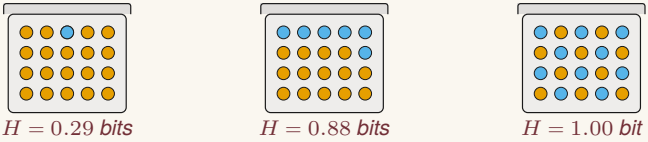


How unpredictable is the next ball?

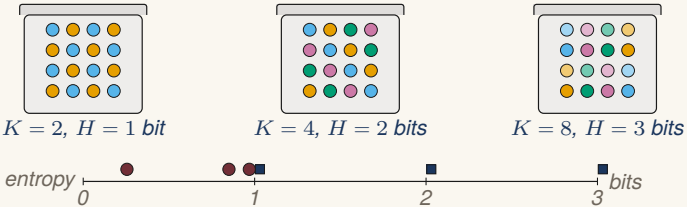
An urn with bias p produces a stream of white and yellow draws. When $p = \frac{1}{2}$ every draw is a coin flip, with maximal surprise. When $p = 0.99$ almost every draw is white, and the sequence is nearly redundant. *Entropy* is the name we give to this gradient between certainty and surprise.

Two ways to make a draw unpredictable

ROW 1 — two colours, vary the shape



ROW 2 — uniform, vary the colour count



Entropy grows when the distribution flattens and when more colours enter the urn. The two effects accumulate on the same scale of bits.

Now switching to:

Demo 5 / 10

Entropy mountain

open card Demo 5 on the companion site

The entropy of an urn

Two-colour urn. Bias p for white, $1 - p$ for yellow:

$$H(p) \;=\; -p \log_2 p \;-\; (1-p) \log_2 (1-p).$$

General urn with K colours. Probabilities $p_1, \dots, p_K$ that sum to 1:

$$H(p_1, \dots, p_K) \;=\; -\sum_{i=1}^{K} p_i \log_2 p_i .$$

No compression scheme can describe an urn's draws in fewer than H bits per ball. Eckart and Young gave the floor for compressing a matrix; entropy is the floor for compressing a sequence.

The two-colour formula is the special case $K = 2$. Both grow with uniformity; the general formula also grows with the number of colours.

II

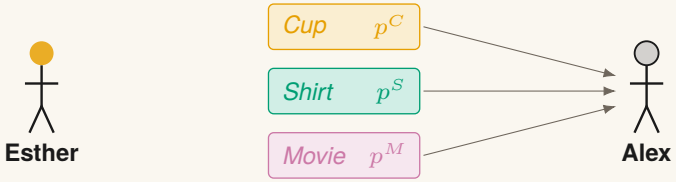
Prediction

"We must choose between alternative actions whose true rewards are not known."

After Robbins, Some aspects of the sequential design of experiments, 1952

Three campaigns

Esther runs an online shop. Three ad campaigns to test, one customer profile, a small budget.



1. How do we estimate p^C, p^S, and p^M?
2. How do we *efficiently* find which is the largest?

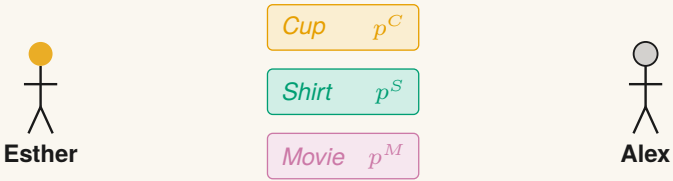
Applications

Bandits show up wherever you must *choose* from a menu of options whose payoffs you do not know.

- *Ad selection.* Which campaign converts most often?
- *Content recommendation.* Which post earns the next click?
- *Resource allocation.* Where does the next dollar buy the most return?
- *Robotics.* Which motion succeeds most reliably?

Same recipe, different names: explore, exploit, learn.

Challenges



1. Neither Esther *nor Alex* knows p^C, p^S, p^M in advance.
2. Showing an ad costs Esther money, so she cannot test forever.

The solution: UCB algorithm

ALGORITHM 1. UPPER CONFIDENCE BOUND (UCB1)

Require: $\alpha > 2, \quad T \in \mathbb{N}$

1: $S_{0,i} = 0, \quad \hat{\mu}_{0,i} = 0, \quad i = 1, \dots, d$
2: **for** $t = 1$ **to** T **do**
3: $\quad A_t = \arg\max_{i=1,\dots,d} \left\{ \hat{\mu}_{t-1,i} + \sqrt{2\alpha \ln t \,/\, S_{t-1,i}} \right\}$
4: $\quad$ observe reward g_{t,A_t}
5: $\quad S_{t,i} \leftarrow S_{t-1,i} + \mathbf{1}[A_t = i]$
6: $\quad$ update $\hat{\mu}_{t,i} = \dfrac{1}{S_{t,i}} \sum_{j=1}^{t} g_{j,A_j} \mathbf{1}[A_j = i]$
7: **end for**

Read this, do not decode it. The rest of Act II builds this algorithm from scratch.

Auer, Cesa-Bianchi & Fischer (2002).

Agenda

1. *Estimating one probability.*

   Customers as urns.

2. *Multiple probabilities and multiple urns.*

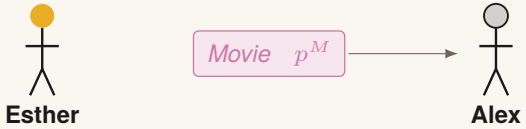
   ε-greedy and UCB.

3. *Many customers and items.*

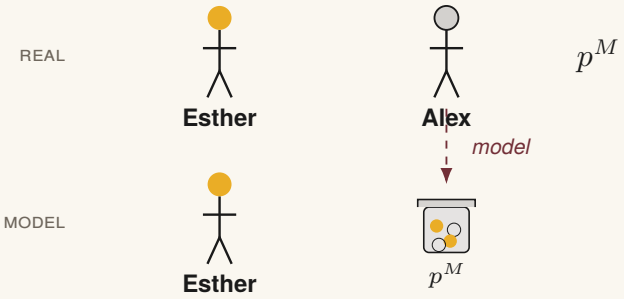
   Contextual urns.

Chapter 1: Estimating one probability

Just one product. Just one probability.

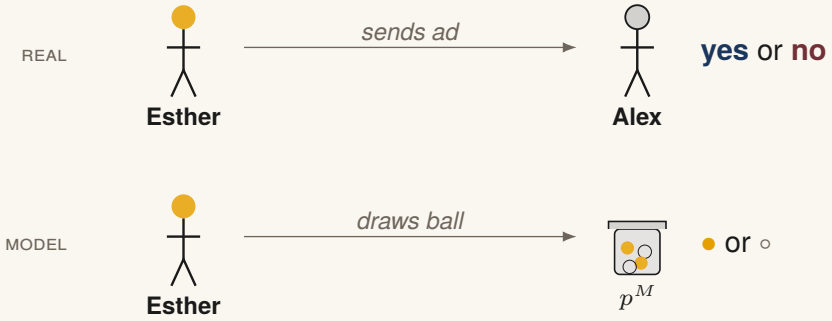


The customer is an urn



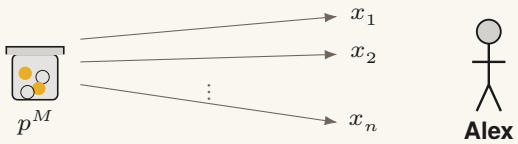
Alex's response distribution is the bias p^M. We model him as an urn whose balls are clicks and non-clicks.

The customer is an urn



$$p^M = \frac{w^M}{n^M}$$

The empirical estimator



($x_i = 1$ if conversion, 0 otherwise.)

$$\hat{p}_n^M = \bar{x}_n = \frac{x_1 + x_2 + \cdots + x_n}{n}$$

The empirical estimator converges

As n grows, $\hat{p}_n^M$ gets closer and closer to p^M with high probability.

$$\hat{p}_n^M \xrightarrow[n\to\infty]{\mathbb{P}} p^M$$

Formally, for any $\delta, \epsilon > 0$, there is $N \in \mathbb{N}$ such that

$$\mathbb{P}\big(|\hat{p}_n^M - p^M| > \delta\big) < \epsilon \quad \text{for } n \geq N.$$

This is an instance of the law of large numbers.

Confidence intervals

Estimating from few balls is risky.

Better: an *interval* instead of a point estimate.

A simple choice:

$$\left(\hat{p}_n - \tfrac{1}{\sqrt{n}},\ \hat{p}_n + \tfrac{1}{\sqrt{n}}\right)$$

sample	*interval*
$n = 5,\ \ \hat{p} = \frac{2}{5}$	$(\frac{2}{5} - \frac{1}{\sqrt{5}},\ \frac{2}{5} + \frac{1}{\sqrt{5}})$
$n = 50,\ \ \hat{p} = \frac{12}{50}$	$(\frac{12}{50} - \frac{1}{\sqrt{50}},\ \frac{12}{50} + \frac{1}{\sqrt{50}})$
$n = 500,\ \ \hat{p} = \frac{124}{500}$	$(\frac{124}{500} - \frac{1}{\sqrt{500}},\ \frac{124}{500} + \frac{1}{\sqrt{500}})$

Same urn, different sample sizes. As n grows, the interval shrinks like $1/\sqrt{n}$.

Now switching to:

Demo 6 / 10

Three-urn casino

open card Demo 6 on the companion site

Agenda

1. *Estimating one probability.*

   Customers as urns.

2. *Multiple probabilities and multiple urns.*

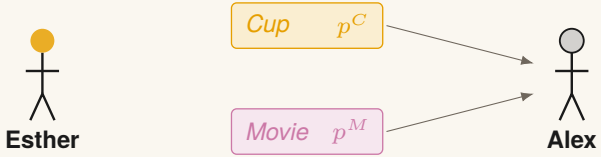
   ε-greedy and UCB.

3. *Many customers and items.*

   Contextual urns.

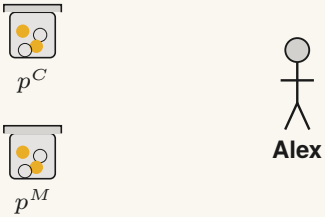
Chapter 2: Two urns

Bring back the cup. Now two products, two probabilities.



Naïve: estimate then exploit

Two urns, one customer, finite budget.



What is the simplest possible algorithm?

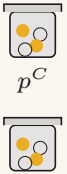
Naïve: estimate then exploit





1. Choose a number n.
2. Compute estimators $\hat{p}_n^C, \hat{p}_n^M$.
3. Exploit the urn with the largest estimate.

Drawbacks!





1. Choose a number n.
2. Compute $\hat{p}_n^C, \hat{p}_n^M$.
3. Exploit the largest.

- If p^C and p^M are close and n is too small, you exploit the *wrong urn*.
- If p^C and p^M are far apart and n is too large, you *waste rounds* estimating the loser.

ε-greedy: a first fix

Don't commit. At each round, with probability:

- $1 - \varepsilon$: *exploit* the urn with the largest current estimate;
- ε: *explore* a random urn.

Most rounds, exploit. A small fraction, explore. The dial ε chooses between the two pressures.

Sutton & Barto, *Reinforcement Learning: An Introduction*, 1998/2018.

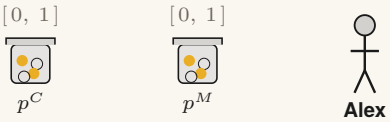
Now switching to:

Demo 7 / 10

Epsilon-greedy walkthrough

open card Demo 7 on the companion site

UCB sampling



1. For each urn u, start with $c_u = [0, 1]$.
2. For $t = 1, 2, \ldots$:
   - 2.1 find the urn $\hat{u}$ with the *highest upper bound*;
   - 2.2 draw a ball x from $\hat{u}$;
   - 2.3 if $x = 1$, get the reward;
   - 2.4 recompute the confidence interval of $\hat{u}$.

Race the algorithms

Four strategies, the same urns, the same hidden biases:

- Pure random: no learning at all.
- Pure greedy: exploit the leader from round one.
- ε-greedy: explore with probability ε.
- UCB: pick the urn with the highest upper bound.

Watch the cumulative regret. Random and greedy grow it *linearly*; ε-greedy and UCB grow it *sublinearly*.

Now switching to:

Demo 8 / 10

Algorithm stadium

open card Demo 8 on the companion site

Agenda

1. *Estimating one probability.*

   Customers as urns.

2. *Multiple probabilities and multiple urns.*

   ε-greedy and UCB.

3. *Many customers and items.*

   Contextual urns.

Chapter 3: Many customers and many items

Real recommenders have millions of items and billions of customers. We cannot run a separate UCB per customer. The fix reuses Act I's matrix factorisation:

1. Factorise historical ratings to obtain an embedding vector z for each item (the step that gave us the book vectors v in Act I).
2. For a new customer u, the preference vector z_u is *unknown*. Estimate it online from the few interactions observed so far, by regression on observed rewards.
3. The urn's bias for offering item z to customer u is the inner product of Act I: $p(z, z_u) = \max(0,\ z^\top z_u)$.
4. Each round, pick the item whose predicted bias has the highest upper bound (UCB on $z^\top \hat{z}_u$).

Compression (Act I) provides z. Prediction (Act II) provides $\hat{z}_u$.

One geometry, two settings

In Act I (recsys), each reader had a vector u. Each book had a vector v. Preference was the inner product:

$$r \;=\; u \cdot v.$$

In contextual bandits, each customer has a vector z_u. Each item has a vector z. The urn's bias is the same inner product:

$$p \;=\; \max(0,\; z^\top z_u).$$

Same matrix factorisation, two different problems.

How many users do you need?

It depends on the gap.

Two urns with conversion rates 0.50 and 0.40 are easy to tell apart. Two urns at 0.50 and 0.49 are nearly indistinguishable; separating them takes *far* more samples.

The required sample size scales roughly as $1/(p_1 - p_2)^2$. Halve the gap, and you need four times the data.

Now switching to:

Demo 9 / 10

Distinguishability dial

open card Demo 9 on the companion site

Lai & Robbins (1985)

Each arm i has a true conversion rate $p_i \in [0, 1]$ (a number). The best arm i^* has rate $p^* = \max_i p_i$. No algorithm beats:

$$\liminf_{T \to \infty} \frac{R_T}{\log T} \geq \sum_{i \neq i^*} \frac{p^* - p_i}{\mathrm{KL}(p_i \,\|\, p^*)}.$$

Both pieces are *numbers*:

- $p^* - p_i$ is the *gap*: how much you lose per round by playing arm i instead of the winner.
- $\mathrm{KL}(p_i \,\|\, p^*)$ is the *Bernoulli KL divergence*, $p_i \log \frac{p_i}{p^*} + (1 - p_i) \log \frac{1-p_i}{1-p^*}$, measuring how distinguishable arm i is from the best.

Near-twins (small gap, small KL) cost you the most.

Two halves, one ceiling

Eckart–Young bounded compression.

Lai–Robbins bounds prediction.

The two halves of this lecture share one ceiling, and we have not yet named the quantity in which both ceilings are written. That is what the Interlude and the Epilogue are for.



“Compression is comprehension.”

Marcus Hutter, on the spirit of his prize

Epilogue: Duality

A perfect predictor is a perfect compressor

Two halves of one subject.

- A *forecaster* assigns $\hat{p}$ to the next symbol; her loss is $-\log \hat{p}$ per draw.
- An *encoder* writes short codewords for likely symbols; his cost is bits per draw.
- Shannon (1948) proved these are the same number, bottoming out at $H(p)$.

Hutter calls this *compression is comprehension*; Delétang et al. (2023) confirm it: a text-trained LM compresses ImageNet better than PNG.

Now switching to:

Demo 10 / 10

Compression-prediction twin

open card Demo 10 on the companion site

Floor + inefficiency

$$\underbrace{-\,\mathbb{E}_p[\log \hat{p}]}_{\text{cross-entropy}} = \underbrace{H(p)}_{\text{Shannon's floor}} + \underbrace{\mathrm{KL}(p \,\|\, \hat{p})}_{\text{your inefficiency}}$$

Recsys: KL is the gap between your factorization and the truth. Bandits: KL is the cost of mistaking one urn for another. Compression: KL is the bits you waste because your predictor is imperfect. One quantity, three roles.

Funes remembered everything and could think nothing.

Machine learning forgets almost everything,
and that is why it understands.

References, I

1. J. L. Borges, “Funes el memorioso,” *Ficciones*, 1944.
2. C. Eckart and G. Young, “The approximation of one matrix by another of lower rank,” *Psychometrika* 1, 211–218, 1936.
3. C. E. Shannon, “A mathematical theory of communication,” *Bell Syst. Tech. J.* 27, 379–423, 1948.
4. T. L. Lai and H. Robbins, “Asymptotically efficient adaptive allocation rules,” *Adv. Appl. Math.* 6, 4–22, 1985.

References, II

5. R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018.
6. M. Hutter, *Universal Artificial Intelligence*, Springer, 2005.
7. G. Delétang *et al.*, "Language modeling is compression," arXiv:2309.10668, 2023.

Image credits. Borges (1951): Grete Stern, public domain. Shannon: Konrad Jacobs, Mathematisches Forschungsinstitut Oberwolfach, CC-BY-SA 2.0 DE. Hutter: Wikimedia Commons (Marcus Hutter), CC-BY-SA. All retrieved from Wikimedia.